COMPUTER ENGINEERING

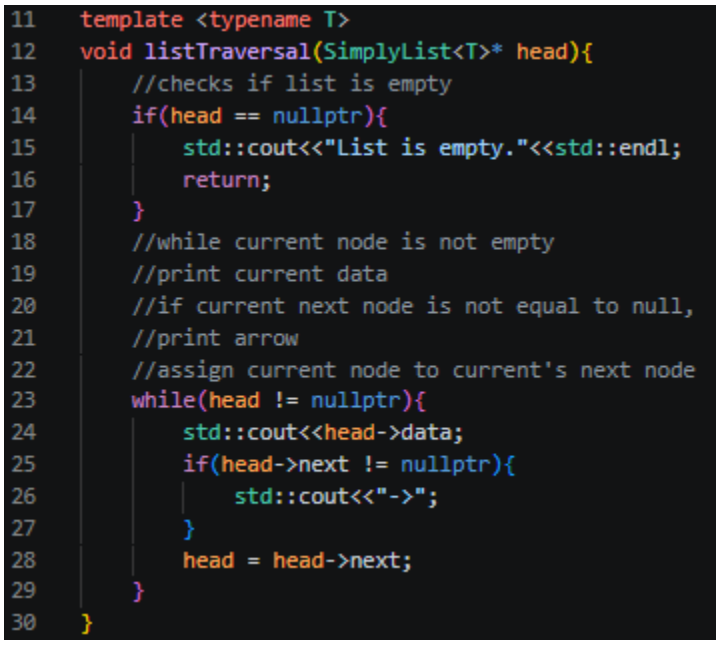
QUEZON CITY

COURSE CODE/TITLE:	CPE010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	Jul 23, 2026	**DATE SUBMITTED:**	Aug 18, 2026
NAME:	Ellaiza C. Vizcayno	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 3 **Linked Lists**

1. PROCEDURE OUTPUT

Screenshot	
Discussion	While the code did not display the output since we did not print it, it actually created a list using a class with its attributes, such as data and the pointer to the next node, initialized. Each node was created using new Node and was assigned a character as its data. The next pointer was then used to connect each node to the next one, forming the linked list C → P → E → 0 → 1 → 0. The last node was assigned nullptr to indicate that there was no next node. However, the code could be improved by adding a way to display or traverse the linked list so that we can verify that the nodes were properly created and connected.

Operation	**Screenshot**
Traversal	

```
11  template <typename T>
12  void listTraversal(SimplyList<T>* head){
13      //checks if list is empty
14      if(head == nullptr){
15          std::cout<<"List is empty."<<std::endl;
16          return;
17      }
18      //while current node is not empty
19      //print current data
20      //if current next node is not equal to null,
21      //print arrow
22      //assign current node to current's next node
23      while(head != nullptr){
24          std::cout<<head->data;
25          if(head->next != nullptr){
26              std::cout<<"->";
27          }
28          head = head->next;
29      }
30  }
```

Insertion at head

```
32  template <typename T>
33  void sllinsertHead(T newData, SimplyList<T>** currentHead){
34      //1. Allocate memory for new node
35      SimplyList<T>* newNode = new SimplyList<T>;
36
37      //2. Put our data into the new node
38      newNode->data = newData;
39
40      //3. Set NExt of the new node to point to the previous head
41      newNode->next = *currentHead;
42
43      //4. Reset head to point to the new node
44      *currentHead = newNode;
45  }
```

Insertion at any part of the list

```
47  template <typename T>
48  void sllGeneralInsert(T newData, SimplyList<T>* prevNode){
49      if (prevNode == nullptr){
50          std::cout<<"Previous Node cannot be Null."<<std::endl;
51          return;
52      }
53      //ALlocate a new node
54      SimplyList<T>* newNode = new SimplyList<T>;
55
56      //store data in the new node
57      newNode->data = newData;
58
59      //point new node to the node previous node was pointing to
60      newNode->next = prevNode->next;
61
62      //point previous node
63      prevNode->next = newNode;
64  }
```

Insertion at the end

```
66  template <typename T>
67  void sllinsertEnd(T newData, SimplyList<T>** head){
68      //allocate new node
69      SimplyList<T>* newNode = new SimplyList<T>;
70
71      //dereference to the head node
72      SimplyList<T>* currentNode = *head;
73
74      //store data in new node
75      newNode->data = newData;
76
77      //point next of new node to NULL
78      newNode->next = nullptr;
79
80      //traverse unil next node is null
81      while(currentNode->next != nullptr){
82          currentNode = currentNode->next;
83      }
84      //point the next of the current node to the new node
85      currentNode->next = newNode;
86  }
```

Deletion of a node

```
87  template <typename T>
88  void sllDeleteNode(T findData, SimplyList<T>** head){
89      //check if there is no list to be deleted
90      if (*head == nullptr) return;
91
92      //find previous node of the node to be deleted
93      SimplyList<T>* currNode = *head;
94      SimplyList<T>* prevNode = nullptr;
95
96      //create searching algorithm/loop
97      while(currNode != nullptr && currNode->data != findData){
98          prevNode = currNode;
99          currNode = currNode->next;
100     }
101     //data was not found
102     if (currNode == nullptr) return;
103
104     //if node is to be deleted is the head node
105     if(prevNode == nullptr){
106         *head = currNode->next;
107     }else{
108         prevNode->next = currNode->next;
109     }
110
111     //we have to delete node
112     delete currNode;
113 }
```

a.

Source Code

main.cpp

```cpp
#include <iostream>
#include "singly_LL.h"

int main(){
    //creating an empty list
    SimplyList<char>* head = new SimplyList<char>;
    SimplyList<char>* second = new SimplyList<char>;
    SimplyList<char>* third = new SimplyList<char>;
    SimplyList<char>* fourth = new SimplyList<char>;
    SimplyList<char>* fifth = new SimplyList<char>;
    SimplyList<char>* sixth = new SimplyList<char>;
    SimplyList<char>* seventh = nullptr;

    //assign the data:
    head->data = 'C';
    head->next = second;

    second->data = 'P';
    second->next = third;

    third->data = 'E';
    third->next = fourth;

    fourth->data = '1';
    fourth->next = fifth;

    fifth->data = '0';
    fifth->next = sixth;

    sixth->data = '1';
    sixth->next = nullptr;

    //Traverse the list by passing the head of the created list into the function
    std::cout<<"original list: "<<std::endl;
    listTraversal(head);

    return 0;
}
```

Console

```
original list:
C->P->E->1->0->1
```

b.

Source Code

```cpp
    //Insert the element 'G' at the start of the list to replace the current node.
    std::cout<<"\ninserting a new node at the head: "<<std::endl;
    sllinsertHead('G', &head);
    listTraversal(head);
```

Console

```
original list:
C->P->E->1->0->1
inserting a new node at the head:
G->C->P->E->1->0->1
```

c.	**Source Code**	`//Insert an element "E" with the previous node element being "P".`
		`std::cout<<"\ninserting a new node at the third: "<<std::endl;`
		`sllGeneralInsert('E', head->next->next);`
		`listTraversal(head);`

	Console	`original list:`
		`C->P->E->1->0->1`
		`inserting a new node at the head:`
		`G->C->P->E->1->0->1`
		`inserting a new node at the third:`
		`G->C->P->E->E->1->0->1`

d.	**Source Code**	`//Delete the node containing the element C`
		`std::cout<<"\ndeleting a node: "<<std::endl;`
		`sllDeleteNode('C', &head);`
		`listTraversal(head);`

	Console	`original list:`
		`C->P->E->1->0->1`
		`inserting a new node at the head:`
		`G->C->P->E->1->0->1`
		`inserting a new node at the third:`
		`G->C->P->E->E->1->0->1`
		`deleting a node:`
		`G->P->E->E->1->0->1`

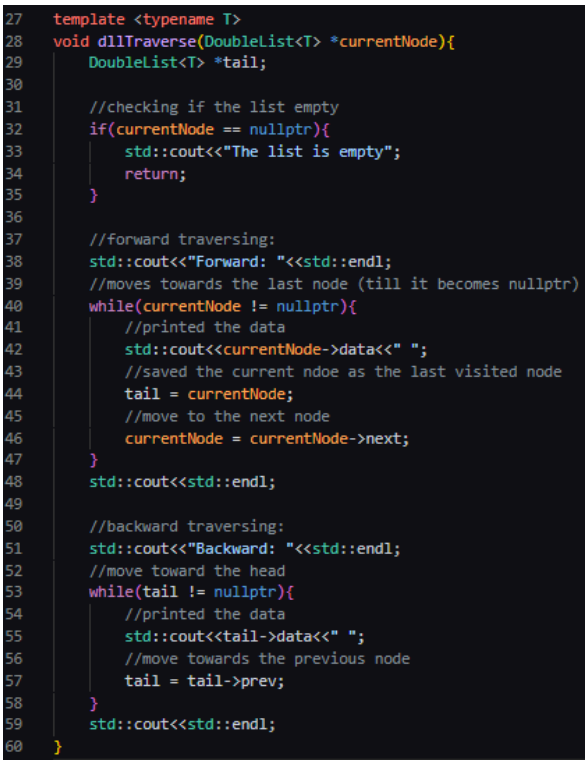
e.	**Source Code**	`//Delete the node containing the element P`
		`std::cout<<"\ndeleting a node: "<<std::endl;`
		`sllDeleteNode('P', &head);`
		`listTraversal(head);`

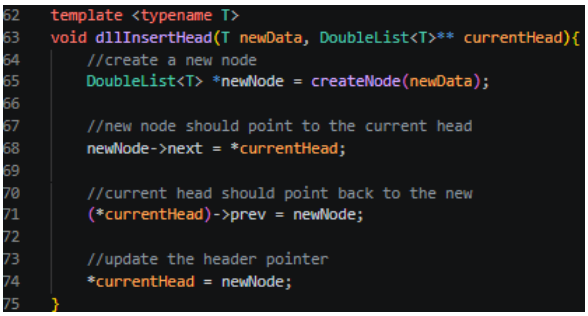
	Console	`original list:`
		`C->P->E->1->0->1`
		`inserting a new node at the head:`
		`G->C->P->E->1->0->1`
		`inserting a new node at the third:`
		`G->C->P->E->E->1->0->1`
		`deleting a node:`
		`G->P->E->E->1->0->1`
		`deleting a node:`
		`G->E->E->1->0->1`

f.	**Source Code**	`//Show the elements in the list. Output should be "GEE101"`
		`std::cout<<"new list: "<<std::endl;`
		`listTraversal(head);`

	Console	`new list:` `G->E->E->1->0->1`

Screenshot(s)	Analysis


``
template <typename T>
void dllTraverse(DoubleList<T> *currentNode){
    DoubleList<T> *tail;
 	
    //checking if the list empty
    if(currentNode == nullptr){
        std::cout<<"The list is empty";
        return;
    }
 	
    //forward traversing:
    std::cout<<"Forward: "<<std::endl;
    //moves towards the last node (till it becomes nullptr)
    while(currentNode != nullptr){
        //printed the data
        std::cout<<currentNode->data<<" ";
        //saved the current ndoe as the last visited node
        tail = currentNode;
        //move to the next node
        currentNode = currentNode->next;
    }
    std::cout<<std::endl;
 	
    //backward traversing:
    std::cout<<"Backward: "<<std::endl;
    //move toward the head
    while(tail != nullptr){
        //printed the data
        std::cout<<tail->data<<" ";
        //move towards the previous node
        tail = tail->prev;
    }
    std::cout<<std::endl;
}
``
	This operation allows you to traverse a doubly linked list, both forward and backwards. The parameter needed for this operation is the head, as when it comes to lists, it looks at the head first. The next line of code is for checking if the list is empty. When it determines that the list is empty, it will print "The list is empty" and return, ending the operation. Now comes the forward traversing. While the currentNode is not equal to nullptr, it will print the data of the current node. Essentially, this means that it will continue going through the list until the current node becomes a null pointer. The current node is also saved as tail, which allows it to remember the last node visited. After reaching the end of the list, the backward traversing starts. It uses tail and moves through the list using the prev pointer. It will continue printing the data of each node until tail becomes nullptr. This allows the list to be traversed backwards, from the last node back to the head.


``
template <typename T>
void dllInsertHead(T newData, DoubleList<T>** currentHead){
    //create a new node
    DoubleList<T> *newNode = createNode(newData);
 	
    //new node should point to the current head
    newNode->next = *currentHead;
 	
    //current head should point back to the new
    (*currentHead)->prev = newNode;
 	
    //update the header pointer
    *currentHead = newNode;
}
``
	This operation is used to insert a node at the start or the head. It uses the data you want to insert and the address of the current head of the list. The operation starts by creating a new node using a function called createNode(newData), with the data intended to be inserted and assigning this to a pointer called newNode, which will hold the address of the new node. The newNode will then point to the address of the current node as the next node. Then, the address of the current head will point back to the new node as the previous node. Finally, the new node will be assigned to the address of the current head, making the new node the new head.

```cpp
template <typename T>
void dllInsertAny(T newData, DoubleList<T> *prevNode){
    //check if it is the head node (previous node is null)
    if (prevNode == nullptr){
        std::cout << "Previous node cannot be null!" << std::endl;
        return;
    }
    //create a new node
    DoubleList<T>* newNode = createNode(newData);

    //point new node to the node previous node was pointing to
    newNode->next = prevNode->next;

    //point previous node to the new node
    newNode->prev = prevNode;

    //if there is a node after prevNode
    //make it point back to the new node
    if (prevNode->next != nullptr){
        prevNode->next->prev = newNode;
    }

    //prevNode now points to the newNode
    prevNode->next = newNode;
}
```

This operation is used to insert data in the middle of a list. Its parameter uses the data used to insert and the location of where it was intended to be inserted. First, it checks if the previous node is empty. If it is null, it will print "Previous node cannot be null!" and return, ending the operation. Then, it creates a new node using createNode(newData). The new node will point to the node that the previous node was pointing to using the next pointer. It will also point back to the previous node using the prev pointer. If there is a node after the previous node, its prev pointer will be updated to point to the new node. Finally, the previous node will point to the new node using its next pointer.

```cpp
template <typename T>
void dllInsertEnd(T newData, DoubleList<T> *currentHead){
    //create new node
    DoubleList<T> *newNode = createNode(newData);
    //traverse to last node
    while(currentHead->next != nullptr){
        currentNode = currentHead->next;
    }
    //connect the last node to the new node
    currentHead->next = newNode;

    //connect the new node back to the last node
    newNode->prev = currentHead;
}
```

When inserting an element at the end of the list, this operation is used. The idea is that this operation will first create a new node. Then, it will traverse until the end of the list and assign the next node of the last node to the new node. Then, it connects the new node back to the last node using the prev pointer.

```cpp
template <typename T>
void dllDeleteNode(T findData, DoubleList<T> **currentHead){
    //check if list is empty
    if(*currentHead == nullptr) return;

    //find previous node of the node to be deleted
    DoubleList<T>* currNode = *currentHead;
    DoubleList<T>* prevNode = nullptr;

    //create searching algorithm/loop
    while(currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }
    //data was not found
    if(currNode == nullptr) return;

    //if node is to be deleted is the head node
    if(prevNode == nullptr){
        *currentHead = currNode->next;

        if(*currentHead != nullptr){
            (*currentHead)->prev = nullptr;
        }
    }else{
        prevNode->next = currNode->next;
        if(currNode->next != nullptr){
        currNode->next->prev = prevNode;
        }
    }

    //we have to delete node
    delete currNode;
}
```

When deleting a node from a list, the operation needs to gather the data intended to delete and the pointer that is pointing to the address of the current head. As always, the first step is to check if the list is empty. Then, temporary nodes currNode and prevNode are created and assigned to the currentHead and nullptr respectively. To search for the data to be deleted, a while loop will be used. This loop will continue traversing the list while currNode is not equal to nullptr and the data of currNode is not equal to the data being searched for. If currNode becomes nullptr, it means that the data was not found and the operation will return. If the node to be deleted is the head node, the currentHead will be updated to the next node. If the next node is going to be the new head, its prev pointer will be set to nullptr. Otherwise, the previous node's next pointer will be connected to the next node. If there is a next node, its prev pointer will then be connected to the previous node. Finally, the node will be deleted from memory.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CODE:

main.cpp

```cpp
#include <iostream>
#include "doubly_LL.h"

void nextSong(DoubleList<std::string>** current);
void prevSong(DoubleList<std::string>** current);
void playAll(DoubleList<std::string>* head);
void addSong(DoubleList<std::string>** head, DoubleList<std::string>** current);
void removeSong(DoubleList<std::string>** head, DoubleList<std::string>** current);
std::string nowPlaying(DoubleList<std::string>* current);

int main(){
    DoubleList<std::string>* head = nullptr;
    DoubleList<std::string>* current = nullptr;
    int choice;

    while(true){
        std::cout << "=====< Playlist >=====\n"
                  << "======================\n";
        std::cout << "Now Playing: " << nowPlaying(current) << "\n"
                  << "======================\n"
                  << "1) Play Next Song\n"
                  << "2) Play Previous Song\n"
                  << "3) Play All Songs\n"
                  << "4) Add Song\n"
                  << "5) Remove Song\n"
                  << "6) Exit\n"
                  << "======================\n"
                  << "Enter Choice: ";
        std::cin >> choice;

        switch(choice){
            case 1:
                std::cout << "======================\n";
                nextSong(&current);
                std::cout << "======================\n";
                break;
            case 2:
                std::cout << "======================\n";
                prevSong(&current);
                std::cout << "======================\n";
                break;
            case 3:
                std::cout << "======================\n";
                playAll(head);
                std::cout << "======================\n";
                break;
            case 4:
                std::cout << "======================\n";
                addSong(&head, &current);
                std::cout << "======================\n";
                break;
            case 5:
                std::cout << "======================\n";
                removeSong(&head, &current);
                std::cout << "======================\n";
                break;
            case 6:
                std::cout << "Exiting..";
                return 0;
```

```
11  int main(){
16      while(true){
31          switch(choice){
60              default:
61                  std::cout << "Invalid choice!" << std::endl;
62              }
63      }
64
65      return 0;
66  }
67
68  void nextSong(DoubleList<std::string>** current){
69      if (*current == nullptr){
70          std::cout << "There is no song in your playlist." << std::endl;
71          return;
72      }
73
74      if ((*current)->next == *current){
75          std::cout << "There is only one song in the playlist." << std::endl;
76          std::cout << "Now Playing: " << (*current)->data << std::endl;
77          return;
78      }
79
80      *current = (*current)->next;
81
82      std::cout << "Now Playing: " << (*current)->data << std::endl;
83  }
84
85  void prevSong(DoubleList<std::string>** current){
86      if (*current == nullptr){
87          std::cout << "There is no song in your playlist." << std::endl;
88          return;
89      }
90
91      if ((*current)->prev == *current){
92          std::cout << "There is only one song in the playlist." << std::endl;
93          std::cout << "Now Playing: " << (*current)->data << std::endl;
94          return;
95      }
96
97      *current = (*current)->prev;
98
99      std::cout << "Now Playing: " << (*current)->data << std::endl;
100 }
101
102 void playAll(DoubleList<std::string>* head){
103     if (head == nullptr){
104         std::cout << "There is no song in your playlist." << std::endl;
105         return;
106     }
107
108     DoubleList<std::string>* current = head;
109
110     std::cout << "\nPlaying all songs:\n";
111
112     do{
113         std::cout << current->data << std::endl;
114         current = current->next;
115     }while(current != head);
```

```cpp
}

void addSong(DoubleList<std::string>** head, DoubleList<std::string>** current){
    std::string song;

    std::cout << "Enter song name: ";
    std::getline(std::cin >> std::ws, song);

    dllInsertEnd(song, head, current);

    std::cout << "Song added successfully!" << std::endl;
}

void removeSong(DoubleList<std::string>** head, DoubleList<std::string>** current){
    if (*head == nullptr){
        std::cout << "There is no song in your playlist." << std::endl;
        return;
    }

    std::string song;

    std::cout << "Enter song name to remove: ";
    std::getline(std::cin >> std::ws, song);

    DoubleList<std::string>* check = *head;
    bool found = false;

    do{
        if(check->data == song){
            found = true;
            break;
        }

        check = check->next;
    }while(check != *head);

    if(!found){
        std::cout << "Song not found." << std::endl;
        return;
    }

    dllDeleteNode(song, head, current);

    std::cout << "Song removed successfully!" << std::endl;
}

std::string nowPlaying(DoubleList<std::string>* current){
    if (current == nullptr){
        return "No song";
    }

    return current->data;
}
```

CODE OUTPUT:

```
=====< Playlist >=====
======================
Now Playing: No song
======================
1) Play Next Song
2) Play Previous Song
3) Play All Songs
4) Add Song
5) Remove Song
6) Exit
======================
Enter Choice: 1
======================
There is no song in your playlist.
======================
```

```
=====< Playlist >=====
======================
Now Playing: No song
======================
1) Play Next Song
2) Play Previous Song
3) Play All Songs
4) Add Song
5) Remove Song
6) Exit
======================
Enter Choice: 2
======================
There is no song in your playlist.
======================
```

```
=====< Playlist >=====
======================
Now Playing: No song
======================
1) Play Next Song
2) Play Previous Song
3) Play All Songs
4) Add Song
5) Remove Song
6) Exit
======================
Enter Choice: 3
======================
There is no song in your playlist.
======================
```

```
=====< Playlist >=====
======================
Now Playing: No song
======================
1) Play Next Song
2) Play Previous Song
3) Play All Songs
4) Add Song
5) Remove Song
6) Exit
======================
Enter Choice: 4
======================
Enter song name: Randomantic by James Reid
Song added successfully!
======================
```

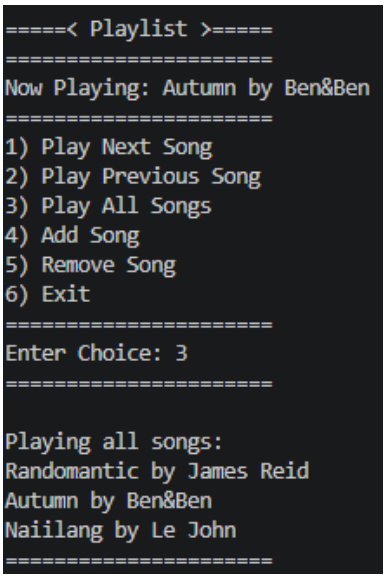
```
=====< Playlist >=====
======================
Now Playing: Randomantic by James Reid
======================
1) Play Next Song
2) Play Previous Song
3) Play All Songs
4) Add Song
5) Remove Song
6) Exit
======================
Enter Choice: 4
======================
Enter song name: Autumn by Ben&Ben
Song added successfully!
======================
```

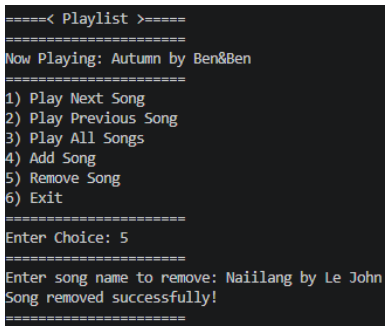
```
=====< Playlist >=====
======================
Now Playing: Randomantic by James Reid
======================
1) Play Next Song
2) Play Previous Song
3) Play All Songs
4) Add Song
5) Remove Song
6) Exit
======================
Enter Choice: 4
======================
Enter song name: Naiilang by Le John
Song added successfully!
======================
```

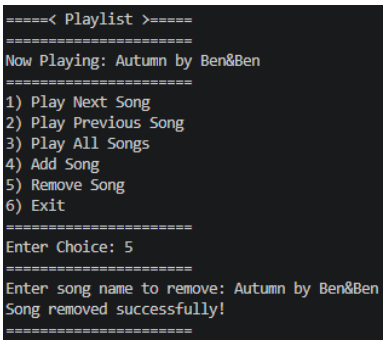
```
=====< Playlist >=====
======================
Now Playing: Randomantic by James Reid
======================
1) Play Next Song
2) Play Previous Song
3) Play All Songs
4) Add Song
5) Remove Song
6) Exit
======================
Enter Choice: 1
======================
Now Playing: Autumn by Ben&Ben
======================
```

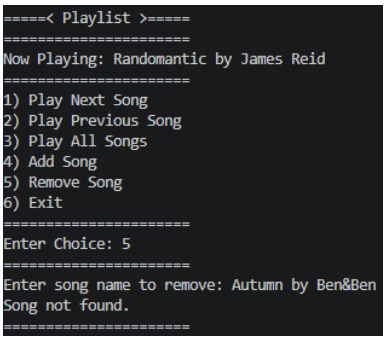
```
=====< Playlist >=====
======================
Now Playing: Autumn by Ben&Ben
======================
1) Play Next Song
2) Play Previous Song
3) Play All Songs
4) Add Song
5) Remove Song
6) Exit
======================
Enter Choice: 1
======================
Now Playing: Naiilang by Le John
======================
```

```
=====< Playlist >=====
======================
Now Playing: Naiilang by Le John
======================
1) Play Next Song
2) Play Previous Song
3) Play All Songs
4) Add Song
5) Remove Song
6) Exit
======================
Enter Choice: 2
======================
Now Playing: Autumn by Ben&Ben
======================
```

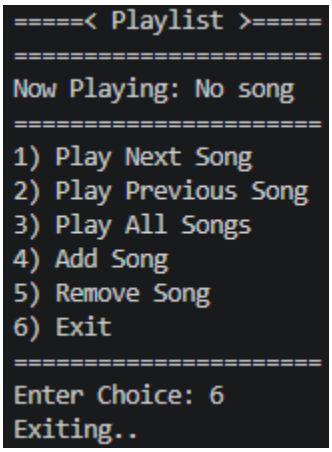

=====< Playlist >=====
Now Playing: Autumn by Ben&Ben
1) Play Next Song
2) Play Previous Song
3) Play All Songs
4) Add Song
5) Remove Song
6) Exit
Enter Choice: 3
Playing all songs:
Randomantic by James Reid
Autumn by Ben&Ben
Naiilang by Le John


=====< Playlist >=====
Now Playing: Autumn by Ben&Ben
1) Play Next Song
2) Play Previous Song
3) Play All Songs
4) Add Song
5) Remove Song
6) Exit
Enter Choice: 5
Enter song name to remove: Naiilang by Le John
Song removed successfully!


=====< Playlist >=====
Now Playing: Autumn by Ben&Ben
1) Play Next Song
2) Play Previous Song
3) Play All Songs
4) Add Song
5) Remove Song
6) Exit
Enter Choice: 5
Enter song name to remove: Autumn by Ben&Ben
Song removed successfully!


=====< Playlist >=====
Now Playing: Randomantic by James Reid
1) Play Next Song
2) Play Previous Song
3) Play All Songs
4) Add Song
5) Remove Song
6) Exit
Enter Choice: 5
Enter song name to remove: Autumn by Ben&Ben
Song not found.


=====< Playlist >=====
Now Playing: Randomantic by James Reid
1) Play Next Song
2) Play Previous Song
3) Play All Songs
4) Add Song
5) Remove Song
6) Exit
Enter Choice: 5
Enter song name to remove: Randomantic by James Reid
Song removed successfully!


=====< Playlist >=====
Now Playing: No song
1) Play Next Song
2) Play Previous Song
3) Play All Songs
4) Add Song
5) Remove Song
6) Exit
Enter Choice: 6
Exiting..

Conclusion

I learned that when working with linked lists, you really have to keep track of the head and tail. In a singly linked list, you mainly need to know what comes next after each node, but in a doubly linked list, you also need to know what came before it. This makes the list more flexible since you can move in both directions, but it also means there are more pointers to keep track of.

At first, the procedure seemed complicated because of the number of pointers and addresses involved. However, the process becomes easier when it is broken down into smaller steps. The list is first established by creating the nodes, after which the next and prev pointers are assigned to establish the relationships between them. The different operations then modify these relationships depending on whether a node needs to be inserted, removed, or accessed. For insertion, the pointers of the surrounding nodes must be updated so that the new node becomes properly connected to the list. For deletion, the connections between the nodes before and after the target node must first be updated before the node is removed. This shows that the nodes themselves do not need to be physically moved; instead, modifying the pointers changes how the nodes are connected and therefore changes the structure of the list.

For the supplementary activity, I applied the doubly linked list structure to a circular playlist. Some parts of the template had to be modified to accommodate the circular structure. In a regular doubly linked list, the last node can point to nullptr, while in a circular list, the last node must point back to the first node. Similarly, the first node's prev pointer must maintain its connection to the last node. This allows the playlist to continuously move through its nodes without reaching nullptr.

Pointers were also used in main.cpp to access the playlist and control the currently playing song. The next pointer of the current node is used when selecting the next song, while the prev pointer is used when selecting the previous song. The current pointer serves as a reference to the node currently being played and needs to be updated whenever navigation or playlist modifications change which node should be accessed. This also shows the difference between the linked-list functions and the main program. The template functions are responsible for modifying the nodes and their connections, while main.cpp handles the playlist behavior and determines which node should be accessed.

I think I did fairly well in this activity. Pointers and addresses are still confusing to me sometimes, but I feel like I get the gist of how they work and how the different parts of the linked list connect. The activity was tricky, especially when working with the circular list, but being able to get the playlist working helped me understand it better. One thing I realized is that I might be trying too hard to visualize everything in my head. Sometimes I focus too much on imagining where every pointer is going instead of just writing down the nodes and their connections. I think this is one of the main things I need to improve on. Writing the structure down first would probably make it easier for me to trace the pointers and debug the code instead of trying to keep everything in my head.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, formatting, summarization, and explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used as a supplementary tool to assist with understanding concepts, organizing ideas, improving grammar and formatting, and providing guidance during the preparation of the activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.